

Boîte à outils interactive

Théorie des langages et automates finis
Structures de données & algorithmes, en C « from scratch »

KAMDOM NGUETCHESSI MERVEILLE PRISCA

Matricule : 23V2060

Université de Yaoundé I – Département d'Informatique
INF3421 : Langages Formels & Compilation
Supervision : Pr. Etienne Kouokam

Année académique 2025–2026

L'objectif du TP

Transformer la théorie du cours en un vrai outil

Le point de départ

- Le cours couvre toute la théorie des langages : mots, automates, lemme d'Arden, minimisation. . .
- Ces notions restent souvent *abstraites* sur le papier.

Ce qui a été fait

- Une bibliothèque **C**, écrite entièrement à la main (aucune dépendance externe).
- Chaque algorithme du cours accessible via un **menu interactif**.
- Chaque opération produit un *résultat textuel* et une *image* de l'automate.

Pourquoi le langage C ?

Un langage impératif et compilé, proche de la machine

Comment fonctionne C

- **Impératif** : une suite d'instructions qui modifient la mémoire.
- **Compilé** : le code source est traduit une fois en langage machine, puis exécuté directement (très rapide).
- **Bas niveau** : contrôle fin de la mémoire (tableaux, pointeurs, bits).

Pourquoi c'est adapté ici

- Manipuler des matrices de transition et des ensembles de bits est direct et rapide.
- Écrire « from scratch » force à comprendre *vraiment* chaque algorithme.
- Un seul bémol : tout est à faire soi-même, y compris le dessin.

Le cœur : les structures de données

Une seule structure pour AFD, AFN et ε -AFN

```
typedef struct {
    int nStates;
    int nSymb;
    char alphabet[MAX_SYMB];
    int hasEpsilon;           // AFD ? AFN ? eps-AFN ?
    unsigned char delta[MAX_STATES]
                          [MAX_SYMB+1]
                          [MAX_STATES];
    unsigned char isInitial[MAX_STATES];
    unsigned char isFinal[MAX_STATES];
} Automaton;
```

Deux choix de conception clés

- **Structure unique** : un drapeau hasEpsilon et une colonne réservée pour les ε -transitions $\Rightarrow$ pas de code dupliqué.
- **Ensembles d'états = entiers 64 bits** (BitSet) : union, appartenance, comparaison deviennent immédiates, sans allocation.
- **Prix à payer** : limite de ≈ 60 états (largement suffisant).

Les algorithmes de construction

De l'expression régulière vers l'automate, et l'inverse

Regex → automate

- **Thompson** : chaque sous-expression = petit automate, assemblé par récursion sur l'arbre syntaxique (avec ε -transitions).
- **Glushkov** : une position par symbole, calcul de *Nullable* / *First* / *Last* / *Follow* ; aucune ε -transition.

Automate → regex

- Lemme d'**Arden** : $X = A \cdot X + B$ a pour solution $A^* \cdot B$.
- Un *seul* moteur (méthode de Gauss) sert à la fois pour résoudre un système d'équations *et* pour extraire la regex d'un automate.
- `regex_simplify` évite l'explosion de la taille des expressions.

Astuce d'ingénierie

Ces deux algorithmes sont mathématiquement identiques : les fusionner en un moteur unique réduit le code *et* supprime une source d'incohérence.

Les algorithmes de transformation

Nettoyer et réduire un automate

- **Déterminisation** (`aut_determinize`) : construction des sous-ensembles, avec ε -fermeture. On ne construit que les états réellement atteints (parcours en largeur), pas les 2^n possibles.
- **Complétion** (`aut_complete`) : ajoute un unique état puits absorbant pour les transitions manquantes.
- **Émondage** (`aut_trim`) : ne garde que les états *utiles* = accessibles (depuis l'initial) et co-accessibles (vers un final).
- **Minimisation** (`aut_minimize`) : algorithme de **Moore**, raffinement itératif de la partition {finaux, non-finaux} jusqu'à stabilité.
- **Automate canonique** : émondage + minimisation $\Rightarrow$ le plus petit automate reconnaissant le langage (Myhill-Nerode), unique.

Les opérations de clôture

Combiner des langages, avec le théorème correspondant

Opération	Fonction	Construction / théorème
Union	aut_union_product	Produit d'AFD complets (Th. 5)
Intersection	aut_intersection	Produit, $F = F_1 \times F_2$ (Th. 6)
Complément	aut_complement	AFD complet, $F' = Q \setminus F$ (Th. 4)
Différence	aut_difference	$A \cap \overline{B}$
Concaténation	aut_concat	ε -transitions finaux $\rightarrow$ initiaux (Th. 8)
Étoile	aut_star	Bouclage ε (Th. 9)
Miroir	aut_mirror	Inversion des arcs (Th. 7)

- Les opérations binaires *unifient d'abord les alphabets*, puis déterminisent et complètent : condition de validité des théorèmes.
- **Équivalence** : $A \equiv B$ ssi $L(A) \setminus L(B)$ et $L(B) \setminus L(A)$ sont vides.

Rendre le tout visible : moteur PGM & interface

Un moteur graphique écrit à la main

- Tracé de segments (**Bresenham**) et de cercles (cercle médian).
- Arcs courbes fléchés pour distinguer deux transitions opposées.
- Police bitmap 5×7 maison, export au format **PGM**.
- Les états sur un cercle, l'initial fléché, les finaux en double cercle.

Une interface interactive

- Un « espace de travail » persistant : automate A , automate B , une regex courante.
- Chaque transformation remplace $A \Rightarrow$ on enchaîne les opérations naturellement.
- 11 rubriques de menu ; chaque saisie est validée avec un message clair.

Validation, tests et difficultés

La validation croisée (la plus forte)

- Pour 8 expressions régulières : Thompson → extraction de regex → Thompson → test d'équivalence.
- **Les 8 cas sont équivalents** $\Rightarrow$ preuve expérimentale que 4 algorithmes indépendants sont cohérents entre eux.
- Minimisation vérifiée à la main (exemple 5 : 4 états $\rightarrow$ 3).

Difficultés rencontrées

- Alphabet « fantôme » : oubli du caractère nul de terminaison (corrigé).
- Choix d'une *structure unique* plutôt que trois (moins de code).
- Un *seul moteur* Arden/Gauss au lieu de dupliquer.
- BitSet plutôt que tableaux dynamiques (plus rapide, moins d'erreurs).

Conclusion comparative

Ce que ce choix d'implémentation apporte

	Approche naïve	Approche retenue
Structure de données	Trois structures (AFD, AFN, ε -AFN)	Une seule , drapeau <code>hasEpsilon</code>
Ensembles d'états	Tableaux dynamiques	BitSet (entier 64 bits)
Extraction de regex	Algorithme dédié (BMC)	Moteur unique Arden/Gauss
Résultat	Code triplé, risque d'incohérence	Code compact, cohérence vérifiée

- De bonnes **structures de données** (structure unifiée, BitSet) simplifient tous les algorithmes qui les utilisent.
- L'outil rend *manipulables, visibles et vérifiables* des concepts abstraits du cours.
- Perfectible (limite de 60 états, PGM minimaliste), mais l'objectif pédagogique est pleinement atteint.

Merci de votre attention — questions & discussion.